Trabajo práctico 18

Gestión y Consolidación Centralizada de Vulnerabilidades con OWASP DefectDojo

Informe de implementación y validación

Materia DevSecOps

Estudiante Ezequiel Cuya

Usuario Git trepiol

Fecha de realización 14 de octubre de 2026
Cierre documental 15 de octubre de 2026

Repositorio https://github.com/trepiol/tp18

Rama de implementación feature/tp18-defectdojo-final

Commit técnico final validado

`84088cbf3125713917ff326069846c851eec4216`

Los documentos y sus evidencias se agregan después de este commit técnico. La ejecución citada corresponde exactamente a este SHA.

Ejecución de validación

GitHub Actions 38664207927

Implementé la centralización real en CE 3.4.101. La importación y la reimportación finalizaron correctamente; el Andon de Trivy siguió bloqueando la publicación y el despliegue. Conservé los hallazgos y las reglas heredadas.

Índice

1 Comandos originales y ejecución real 3

2 Evidencias de ejecución 5

3 Desviaciones respecto de la guía 12

4 Propuesta de versión 2 del enunciado 14

5 Interpretación de mensajes reales 16

6 Novedades y comportamientos observados 17

7 Conclusión y relación con la teoría 18

Resultado comprobado

Instalé DefectDojo en la VM debian, importé doce reportes de cinco herramientas y comprobé la jerarquía por UI y API. La primera colección produjo 12 Tests y 306 Findings. El run técnico posterior también produjo 306 entradas de escáneres; Dojo acumuló 469 registros al conservar dos versiones del Target de imagen. Reimportar los mismos reportes mantuvo 469 → 469 y los mismos IDs y estados.

El run 38664207927 terminó globalmente en failure por Trivy y el lint histórico TP06. El build, los tests, las auditorías informativas, el colector y DefectDojo pasaron. Release y deploy quedaron skipped. Consideré correcto ese comportamiento: gestionar evidencia no autoriza liberar una imagen rechazada por seguridad.

Cómo leer las evidencias

Diferencié la importación inicial del run 38512049385 y la validación CI del run 38664207927. Conservé las capturas originales en entrega/capturas y los comprobantes sanitizados en entrega/evidencias. Los recortes de presentación dentro de Word sólo enfocan tablas reales; los archivos originales permanecen completos.

La validación técnica corresponde al 14 de octubre en Argentina UTC−3 y al 15 de octubre UTC en GitHub. Cerré la documentación y completé las capturas el 15 de octubre en Argentina. Los conteos son observaciones de esas ejecuciones y no una promesa sobre futuros escaneos.

Alcance

Trabajé exclusivamente en TP18. No remedié vulnerabilidades, no modifiqué las dependencias Python, los manifests, las excepciones Gitleaks ni los jobs Trivy y Semgrep. La integración a main queda sujeta a la revisión y autorización de cierre solicitadas.

1 Comandos originales y ejecución real

Tomé como referencia el documento TP18 DefectDojo Vulnerability Management entregado por la cátedra. Comparé sus ejemplos con el repositorio real antes de ejecutar. Mantuve sus cuatro fases y adapté los comandos que no coincidían con las rutas, versiones y condiciones de seguridad existentes.

Instalación indicada por la cátedra

```
git clone https://github.com/DefectDojo/django-DefectDojo.git
cd django-DefectDojo
./dc-build.sh
./dc-up-d.sh postgres-redis
docker compose ps
docker compose logs initializer | grep "Admin password:"
```

No ejecuté esos helpers antiguos ni expuse la contraseña en logs publicables. Consulté el Compose oficial de la release 3.4.101 y guardé la adaptación reproducible en infra/defectdojo/compose.yml. Utilicé imágenes oficiales precompiladas fijadas por digest para evitar construir el stack en una VM con disco limitado. El wrapper exige un entorno privado con permisos restringidos.

```
cd /home/alumno/tp18
bash scripts/preparar-defectdojo.sh
python3 scripts/inicializar-defectdojo.py
bash scripts/defectdojo-compose.sh ps -a
```

Ejecuté este procedimiento y verifiqué la instalación. El initializer terminó con exit 0. PostgreSQL, uWSGI y Valkey quedaron saludables; nginx, Celery worker y beat permanecieron en ejecución. La release oficial utiliza Valkey como broker compatible con Redis.

Configuración y jerarquía

La guía propone crear el contexto desde la UI. Automaticé su creación idempotente y después lo comprobé también en la interfaz: Product Type Aplicaciones Web DevSecOps, Product Notes App y Engagement CI/CD Automated Security Audits en estado In Progress. Obtuve IDs reales 2, 1 y 1; no supuse PRODUCT_ID=1. En esta release la UI usa Organization y Asset para las dos primeras entidades.

Creé Tests con parsers nativos para cada herramienta. Separé Trivy por banda de severidad y por SCA, Container, Secrets e IaC: ocho Tests Trivy más cuatro de las otras herramientas, doce en total.

Comandos originales del pipeline y sus reemplazos

```
docker build -t ${{ env.DOCKER_IMAGE }}:${{ github.sha }} ./backend
helm template ${{ env.HELM_RELEASE_NAME }} ./devops-portfolio \
  -f devops-portfolio/values-prod.yaml > manifests-rendered-prod.yaml
gitleaks detect --source . --report-format json \
  --report-path reports-json/gitleaks-report.json || true
```

Estos son ejemplos originales, no comandos de una nueva ejecución. Conservé el build único heredado en devops-tp12/app/backend y el chart real en devops-tp12/chart con devops-tp12/values-prod.yaml. El historial Gitleaks usa 8.30.0, git --log-opts=--all, configuración existente y redacción; no agregué || true. Dejé el workflow operativo en .github/workflows/cicd.yml.

Sustituí el ejemplo curl con grep de la guía por scripts/defectdojo_api.py y el wrapper estricto scripts/verificar-defectdojo.sh. Verifiqué HTTP 201, consulté cada Test y comparé snapshots. El token se leyó desde un archivo privado o un Secret de GitHub; no quedó escrito en comandos versionados.

```
export DEFECTDOJO_URL=http://127.0.0.1:8080
export DEFECTDOJO_TOKEN_FILE="$PWD/.security-private/defectdojo/api-token"
bash scripts/verificar-defectdojo.sh check
```

El Product Type ID se toma del context.json privado generado por la inicialización. Los comandos completos de importación y reimportación están en la sección 4 y en docs/TP18-GUIA-V2.md.

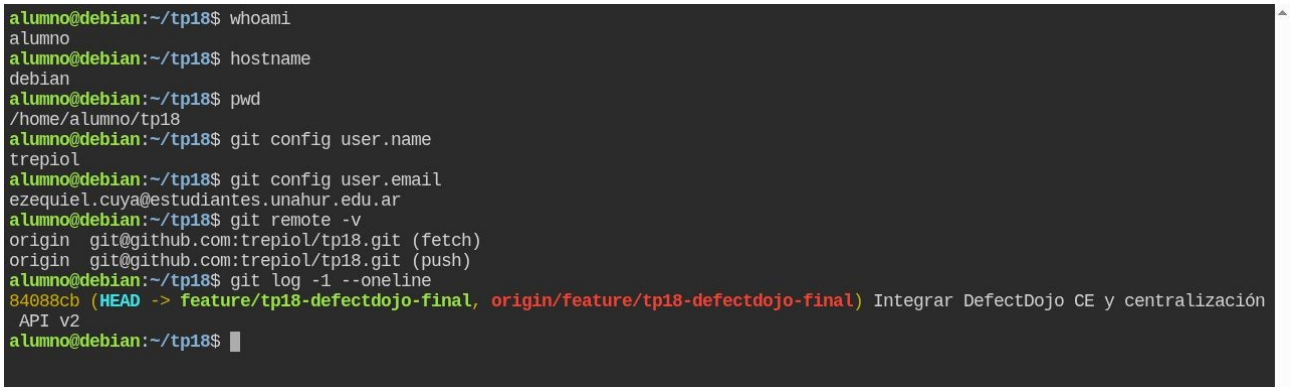


Figura 1. Identidad real de la VM y del repositorio

Ejecuté whoami, hostname, pwd y consultas Git en una terminal Bash real de la VM. El prompt alumno@debian, el origin trepiol/tp18 y el commit 84088cb vinculan las evidencias con mi entorno y la rama de trabajo. No modifiqué otros repositorios.

2 Evidencias de ejecución

Registré capturas auténticas de la terminal de la VM, DefectDojo y GitHub Actions. Los comandos mostrados se ejecutaron en el entorno real y las respuestas API se comprobaron antes de incorporar las imágenes. Ninguna captura contiene tokens, contraseñas o claves privadas.

```
alumno@debian:~/tp18$ bash scripts/defectdojo-compose.sh ps -a --format '{{.Servi
ce}} {{.Status}}'
celerybeat Up 9 hours
celeryworker Up 9 hours
initializer Exited (0) 9 hours ago
nginx Up 9 hours
postgres Up 9 hours (healthy)
uwsgi Up 9 hours (healthy)
valkey Up 9 hours (healthy)
alumno@debian:~/tp18$
```

Figura 2. Stack DefectDojo en Docker Compose

El listado muestra siete servicios: initializer Exited 0 y los seis servicios persistentes activos. Complementé esta observación con PONG de Valkey, pong de Celery y la consulta de salud de los contenedores. No hubo OOMKilled.

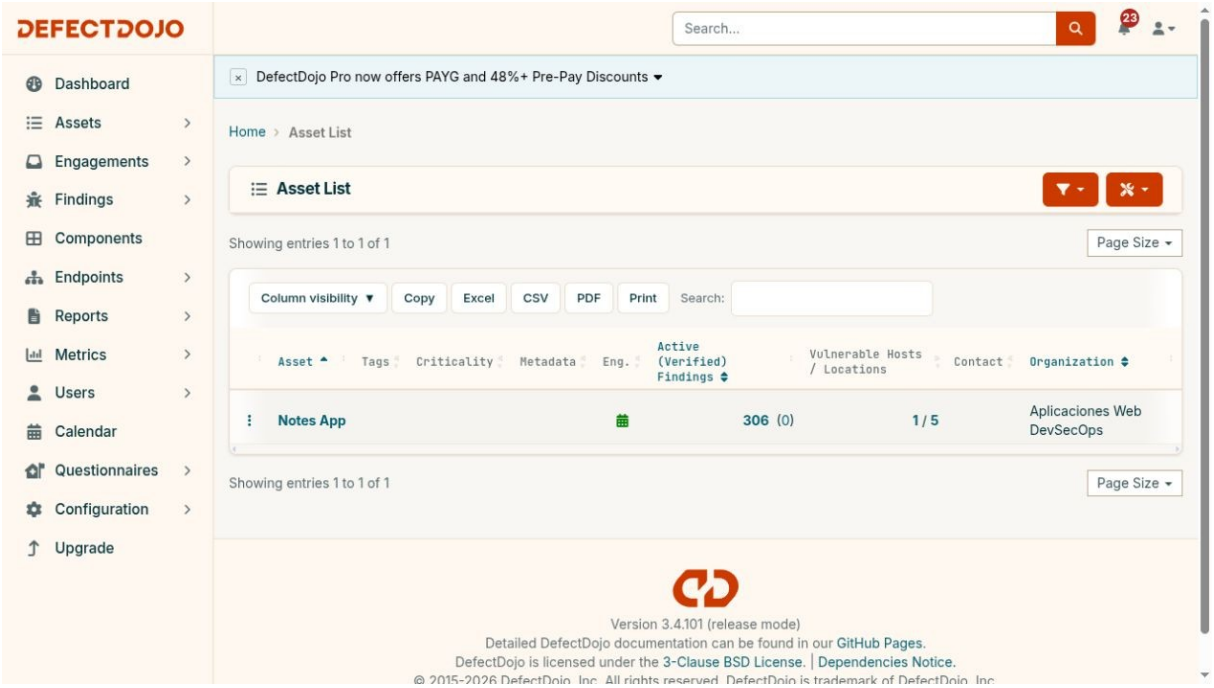


Figura 3. Notes App y su categoría en la interfaz

La captura inicial muestra Notes App asociada a Aplicaciones Web DevSecOps y 306 Findings tras importar el run 38512049385. Este valor pertenece a esa prueba inicial; el estado acumulado posterior de 469 se documenta por separado.

Importación inicial y reimportación del mismo conjunto

```
alumno@debian:~/tp18$ cat .validation/tp18-final/import-inicial.log
API authentication and hierarchy OK: Product Type 2; Product 1; Engagement 1
Collection: 6/6 bundles; missing/invalid=0
OK import-scan: TP18 zap DAST; HTTP 201; Test 1
OK import-scan: TP18 threagile Threat Modeling; HTTP 201; Test 2
OK import-scan: TP18 semgrep SAST; HTTP 201; Test 3
OK import-scan: TP18 gitleaks Git History; HTTP 201; Test 4
OK import-scan: TP18 trivy-high SCA; HTTP 201; Test 5
OK import-scan: TP18 trivy-high Container; HTTP 201; Test 6
OK import-scan: TP18 trivy-high Secrets; HTTP 201; Test 7
OK import-scan: TP18 trivy-high IaC; HTTP 201; Test 8
OK import-scan: TP18 trivy-low SCA; HTTP 201; Test 9
OK import-scan: TP18 trivy-low Container; HTTP 201; Test 10
OK import-scan: TP18 trivy-low Secrets; HTTP 201; Test 11
OK import-scan: TP18 trivy-low IaC; HTTP 201; Test 12
DefectDojo: 12/12 reports; Tests=12; Findings=306; active=306; duplicates=0
alumno@debian:~/tp18$
```

Figura 4. Doce importaciones reales mediante API v2

El registro de scripts/verificar-defectdojo.sh muestra HTTP 201 en import-scan para Tests 1 a 12. El colector validó 6/6 bundles del run 38512049385, SHA 7e39cc4. La consulta posterior confirmó 306 Findings activos y cero duplicados clasificados por Dojo.

```
alumno@debian:~/tp18$ cat .validation/tp18-final/reimport-identico.log
API authentication and hierarchy OK: Product Type 2; Product 1; Engagement 1
Collection: 6/6 bundles; missing/invalid=0
OK reimport-scan: TP18 zap DAST; HTTP 201; Test 1
OK reimport-scan: TP18 threagile Threat Modeling; HTTP 201; Test 2
OK reimport-scan: TP18 semgrep SAST; HTTP 201; Test 3
OK reimport-scan: TP18 gitleaks Git History; HTTP 201; Test 4
OK reimport-scan: TP18 trivy-high SCA; HTTP 201; Test 5
OK reimport-scan: TP18 trivy-high Container; HTTP 201; Test 6
OK reimport-scan: TP18 trivy-high Secrets; HTTP 201; Test 7
OK reimport-scan: TP18 trivy-high IaC; HTTP 201; Test 8
OK reimport-scan: TP18 trivy-low SCA; HTTP 201; Test 9
OK reimport-scan: TP18 trivy-low Container; HTTP 201; Test 10
OK reimport-scan: TP18 trivy-low Secrets; HTTP 201; Test 11
OK reimport-scan: TP18 trivy-low IaC; HTTP 201; Test 12
DefectDojo: 12/12 reports; Tests=12; Findings=306; active=306; duplicates=0
alumno@debian:~/tp18$
```

Figura 5. Reimportación idéntica sin crecimiento

Volví a enviar exactamente esos reportes mediante reimport-scan. Se conservaron 12 Tests, 306 Findings y sus estados. Esta prueba demuestra idempotencia para el mismo conjunto; no prueba deduplicación global entre herramientas o entre Targets diferentes.

Después ejecuté una prueba CI con un SHA nuevo. El pipeline reutilizó los mismos doce Tests y volvió a reimportar sus reportes para verificar estabilidad. Los comprobantes de ambos pasos se publicaron como artifact tp18-defectdojo-84088cb… con retención de catorce días.

Tests de Gitleaks Semgrep y Threagile

Title / Type	Date	Lead	Total Findings	Active (Verified / Fixable)	Mitigated	Duplicates
TP18 gitleaks Git History (Gitleaks Scan) run-38664207927	Oct. 15, 2026 - Oct. 15, 2026		0	0 (0 / 0)	0	0
TP18 semgrep SAST (Semgrep JSON Report) run-38664207927	Oct. 15, 2026 - Oct. 15, 2026		20	20 (0 / 0)	0	0
TP18 threagile Threat Modeling (Threagile risks report) run-38664207927	Oct. 15, 2026 - Oct. 15, 2026		42	42 (0 / 0)	0	0

Figura 6. Resultados de tres herramientas en el Engagement

La tabla real muestra el run 38664207927 en los Tests: Gitleaks 0, Semgrep 20 y Threagile 42. Cero Gitleaks significa que el historial no presentó hallazgos no exceptuados; no fabriqué un secreto para llenar el Test. Los tres riesgos aceptados provienen del modelo Threagile original y se conservaron.

Comprobé que cada Test conserve su tipo de parser y que el cliente API registre build_id, commit_hash y branch_tag en el Test correspondiente. La tarjeta lateral del Engagement puede mostrar Not Specified porque esos metadatos se registran por Test, no como sustituto del contexto común.

Los Findings importados no equivalen a vulnerabilidades confirmadas por un analista. Permanecen sin verificación manual; la plataforma centraliza evidencias y estados para su posterior evaluación.

ZAP y Trivy en el mismo contexto

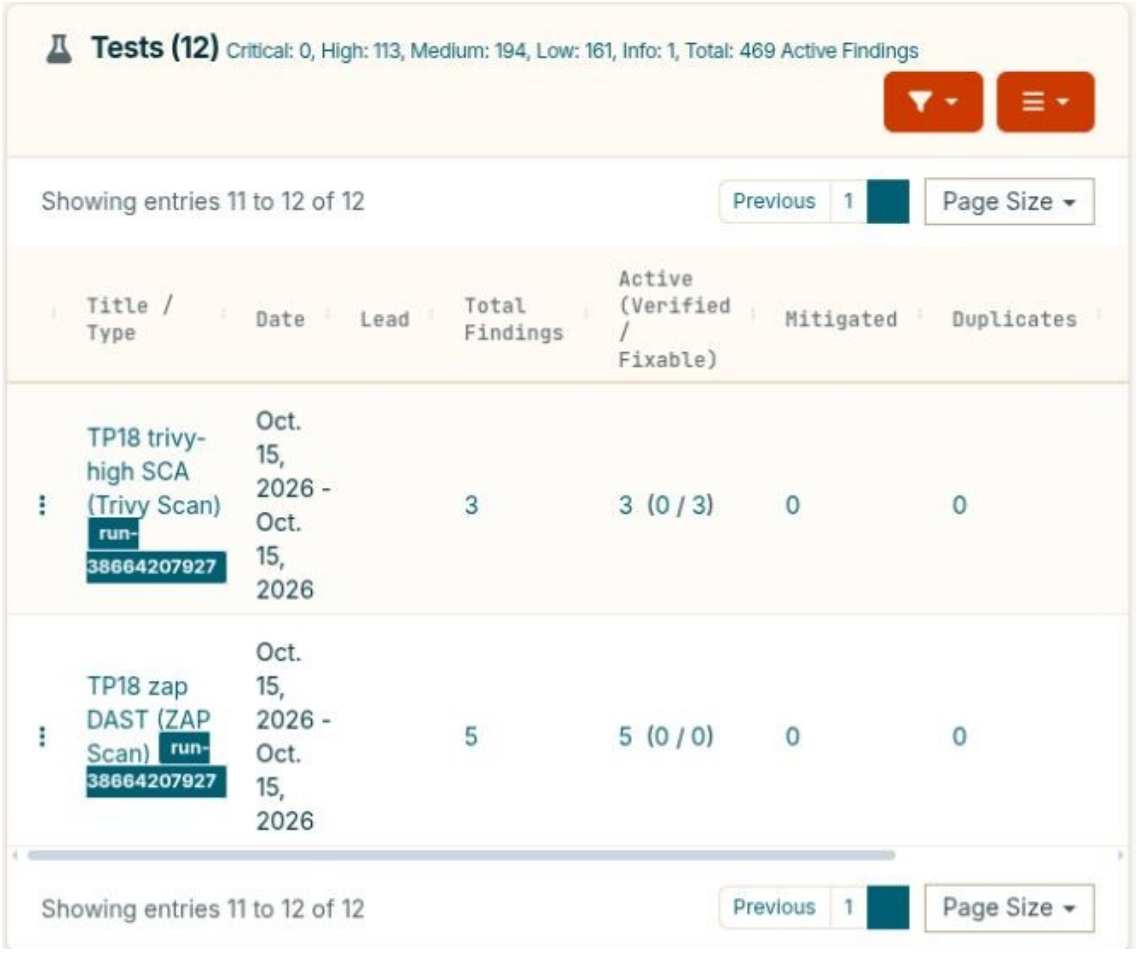


Figura 7. ZAP y Trivy SCA con el total acumulado del Engagement

La interfaz muestra 12 Tests y 469 Findings tras procesar las dos versiones. ZAP conserva 5 alertas y Trivy SCA HIGH/CRITICAL 3. Los restantes Tests Trivy se separan por alcance y banda para no mezclar código, imagen, secretos e infraestructura.

Usé ZAP Scan con XML, Threagile risks report con risks.json, Semgrep JSON Report con JSON nativo sanitizado, Trivy Scan con los JSON por alcance y Gitleaks Scan con JSON redactado. La aceptación real por estos parsers se comprobó mediante HTTP y consulta API; el harness local con stubs sólo aporta una validación adicional de formato.

La matriz de la sección 3 permite comparar entradas por ejecución y registros acumulados por Test. No interpreté 469 como el resultado de un único escaneo ni como cantidad de vulnerabilidades únicas.

Integración real con GitHub Actions



Figura 8. Job DefectDojo aprobado en el run técnico

GitHub muestra Fase 3 Importar y reimportar en DefectDojo en success, con duración de 1 min 9 s. Pasaron la comprobación API, la sincronización, la reimportación idéntica y la publicación de comprobantes.

```
alumno@debian:~/tp18$ gh run view 38664207927 --repo trepiol/tp18 --json jobs --jq '.jobs[] | [.name,.conclusion] | @tsv'
TP17 — Andon Cord Gitleaks      success
Lint histórico TP06     failure
Tests de transformación y recopilación de reportes      success
Fase 2 — Semgrep / SAST multilenguaje y Andon   success
Tests App Integrada TP12        success
Fase 2 — Threagile      success
TP18 — Auditoría histórica Gitleaks     success
Fase 1 — Build una vez y empaquetar     success
Fase 2 — OWASP ZAP aislado      success
Fase 2A — Andon Cord Trivy      failure
Fase 2B — Auditoría Trivy LOW/MEDIUM    success
Fase 3 — Validar bundles de auditoría   success
Fase 3 — Importar y reimportar en DefectDojo    success
Fase 4 — Publicación y deploy Helm      skipped
alumno@debian:~/tp18$
```

Figura 9. Estados completos del run consultados con GitHub CLI

El listado real conserva failure de Trivy y del lint histórico TP06; build, auditorías informativas, colector y DefectDojo pasan. El deploy queda skipped. No publiqué ni desplegué una imagen rechazada por el Andon.

El runner JIT ejecutó únicamente el job revisado y luego se eliminó su registro. No quedó un runner permanente conectado al repositorio público. Cada nueva ejecución requiere revisar su SHA y lanzar un nuevo runner autorizado.

Pruebas del cliente y de la aplicación heredada

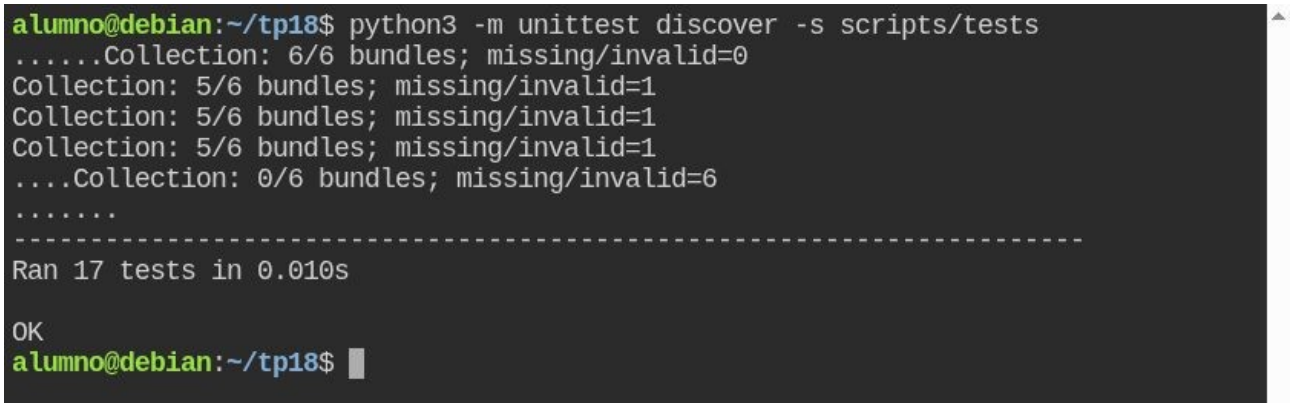


Figura 10. Diecisiete pruebas de scripts aprobadas

Ejecuté unittest discover sobre scripts/tests: once pruebas de transformación y recopilación más seis del cliente API. Los mensajes 5/6 o missing/invalid visibles corresponden a casos negativos intencionales de las fixtures; el colector del run real validó 6/6.

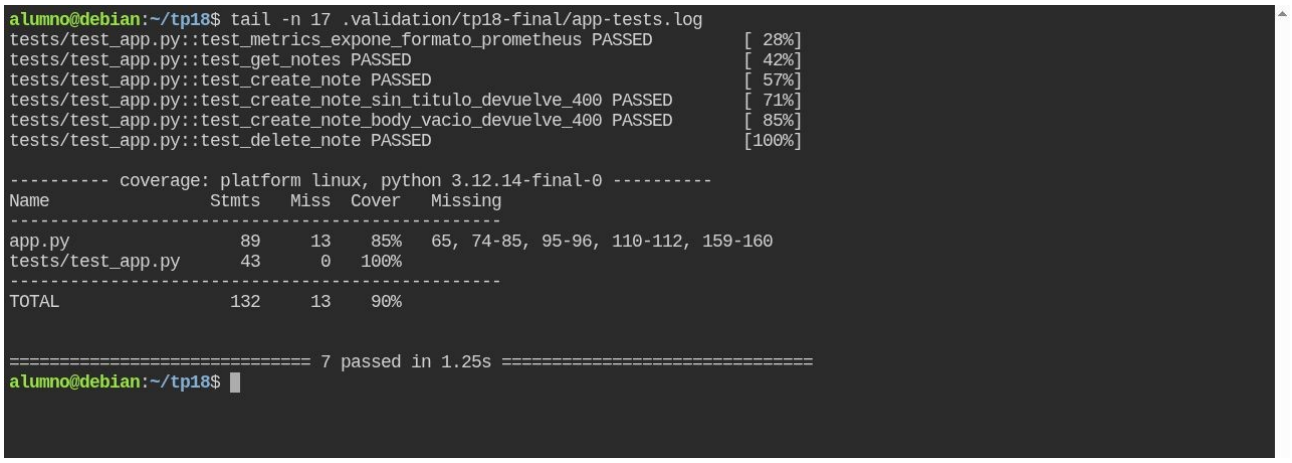


Figura 11. Siete tests de Notes App y cobertura

La ejecución con Python 3.12.14 aprobó los siete tests existentes. La cobertura total fue 90 % y la de app.py 85 %. Conservé requirements y lógica de la aplicación; no actualicé paquetes para modificar los resultados de Trivy.

También pasaron actionlint, YAML, Bash, Helm y Gitleaks. Comparé el render Helm de producción con la referencia y confirmé que no cambió. El clon nuevo del repositorio aprobó las verificaciones documentadas sin recibir los archivos privados de la instancia.

Persistencia y fallos operativos comprobados

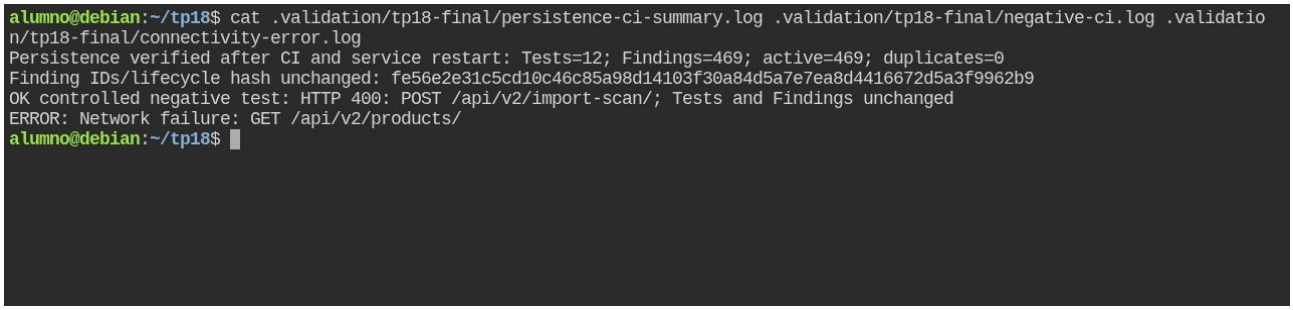


Figura 12. Reinicio y pruebas negativas sin pérdida de datos

Tras respaldar y reiniciar seis servicios, la API mantuvo 12 Tests y 469 Findings activos. El hash de IDs y estados no cambió. La solicitud inválida recibió HTTP 400 sin modificar datos; el puerto cerrado produjo un error de red real con salida 1.

El hash SHA256 de IDs y estados antes y después fue el siguiente:

`fe56e2e31c5cd10c46c85a98d14103f30a84d5a7e7ea8d4416672d5a3f9962b9`

Validé un respaldo PostgreSQL custom mediante pg_restore --list, el archivo de media mediante tar y los checksums. Conservé las claves de configuración en el respaldo privado. No ejecuté una restauración destructiva sobre la base en uso y no configuré un cron de backups: la política propuesta es diaria con siete copias, sujeta a la capacidad disponible.

Comprobantes y trazabilidad

Guardé metadatos sanitizados de la ejecución, manifests con hashes, resultados de API y logs de validación en entrega/evidencias. Excluí tokens, dumps, archivos .env, reportes crudos y diagnósticos privados. Los artifacts de GitHub expiran; los comprobantes publicables preservan el resultado de la validación técnica.

Run técnico 38664207927

Job DefectDojo 116193051011

Commit técnico validado 84088cb

Las capturas se obtuvieron mediante acceso autorizado a la UI y una terminal web temporal enlazada por SSH a loopback. Ese acceso de captura se retira al finalizar; no forma parte del servicio permanente ni del pipeline.

3 Desviaciones respecto de la guía

Documenté las diferencias respaldadas por archivos y ejecuciones. No usé la guía como autorización para debilitar los controles heredados. Cada adaptación responde a la versión instalada, al repositorio existente o a un error observado.

Instalación y rutas

Original: ./dc-build.sh y ./dc-up-d.sh postgres-redis. Problema: el ejemplo no fija una release ni las imágenes y propone un build local costoso. Utilicé preparar-defectdojo.sh y el Compose de CE 3.4.101 con digests. Resultado: siete servicios, initializer exit 0, tres volúmenes persistentes y nginx sólo en 127.0.0.1:8080. Valkey reemplaza a Redis según el stack oficial seleccionado.

Original: docker build ./backend y helm template ./devops-portfolio. Problema: esas rutas no corresponden a TP18. Conservé el backend y chart devops-tp12 y la lógica Build Once Test Everywhere. Resultado: un único tarball por SHA, consumido por ZAP y Trivy; render Helm idéntico al heredado.

Parsers y comandos de escaneo

Original: Threagile con Generic CI/CD Audit y todos los reportes como JSON. Utilicé Threagile risks report y ZAP Scan con XML, comprobados en la instancia. Resultado: Tests de 42 riesgos y 5 alertas. No importé manifests como si fueran reportes de escáner.

Original: Gitleaks 8.18.2 con detect y || true. Conservé la integración 8.30.0 validada, historial completo y redacción. El Andon sigue siendo bloqueante y la auditoría distingue findings de fallos operativos. Resultado: Test vacío válido, sin agregar excepciones ni secretos.

API permisos y conectividad

Original: action de terceros import-scan-action y curl con PRODUCT_ID=1 y grep scan_type. Implementé API v2 directa con campos comprobados, IDs obtenidos del servidor, HTTP 201 y consulta del Test. Resultado: doce importaciones y reimportaciones verificadas, con snapshots de IDs y flags.

Una primera creación de cuenta con el User genérico de Django produjo TypeError de Dojo_User. Deshabilité esa cuenta auxiliar y utilicé el modelo propio de Dojo. La cuenta API definitiva no tiene contraseña utilizable, staff ni superuser; sólo pertenece a Notes App. CE permite editar/importar dentro del producto, no un permiso granular exclusivamente de importación.

Los runners hospedados no alcanzan el loopback de la VM. Utilicé un runner JIT desechable, un job, environment protegido y SHA revisado; excluí PR no confiables. No monté Docker socket, SSH keys ni carpetas del host. Es una operación asistida por el propietario, no un runner público permanente.

Diferencia observada en deduplicación entre imágenes

La colección del run 38664207927 contiene 306 entradas, igual cantidad que la referencia pero con procedencia propia. Dojo conserva 469 registros al unir ambas versiones. Verifiqué que el parser Trivy incluye Target en description y que description participa en el hash nativo junto con title, severity, vulnerability_ids y cwe.

El Target del sistema operativo contiene la etiqueta de imagen con SHA. Cambiar 7e39cc4 por 84088cb altera la descripción de 44 entradas HIGH y 119 LOW/MEDIUM: 163 registros adicionales. Comparé pares con igual CVE, paquete y versión; la descripción coincidió al sustituir sólo el SHA en la comparación. No modifiqué los reportes ni las identidades guardadas.

Test	Herramienta y alcance	Run 386…	Dojo acumulado
1	zap DAST	5	5
2	threagile Threat Modeling	42	42
3	semgrep SAST	20	20
4	gitleaks Git History	0	0
5	trivy-high SCA	3	3
6	trivy-high Container	47	91
7	trivy-high Secrets	0	0
8	trivy-high IaC	11	11
9	trivy-low SCA	6	6
10	trivy-low Container	130	249
11	trivy-low Secrets	0	0
12	trivy-low IaC	42	42

Total por colección: 306. Total acumulado: 469 activos; High 113, Medium 194, Low 161, Info 1 y Critical 0. Dojo marca duplicados 0 y conserva 3 riesgos aceptados. Estos flags no prueban ausencia de repetición semántica entre Targets. La reimportación idéntica del nuevo conjunto mantuvo 469 → 469 y los mismos estados.

Mantuve close_old_findings y reactivación automáticos deshabilitados para preservar los estados reales. Como mejora futura propongo evaluar una identidad estable por contenido conservando las evidencias originales; no alteré la configuración de deduplicación para mejorar artificialmente el total.

4 Propuesta de versión 2 del enunciado

Propongo que una V2 de la consigna fije versiones, formatos y criterios de aceptación por fase. Los comandos siguientes corresponden a esta implementación reproducible; RUTA, SHA_COMPLETO y RUN_ID son parámetros del run elegido, no resultados simulados.

Preparación de la VM y del repositorio

```
git clone git@github.com:trepiol/tp18.git
cd tp18
git switch feature/tp18-defectdojo-final
git status --short --branch
nproc
free -h
df -h /
docker ps
docker volume ls
ss -ltn
```

Comprobé 4 vCPU y 12 GiB RAM. Reservé al menos 8 GiB libres antes de instalar y 4 GiB antes de lanzar el runner. Son presupuestos de este laboratorio, no mínimos oficiales universales. Si no hay espacio para imágenes, base y backups, la guía debería exigir ampliar disco y detenerse; no autorizar prune ni borrar volúmenes ajenos.

Prerrequisitos: Docker con Compose, Git, gh autenticado, Bash, Python 3.11+ con PyYAML, Helm 3.18.4, actionlint y Gitleaks 8.30.0. Para la app utilicé Python 3.12 y requirements heredados. Las versiones/digests de escáneres están en security/tools.json; la infraestructura Dojo está fijada en compose.yml.

Aceptación de build auditorías e instancia

```
python3 -m unittest discover -s scripts/tests -v
python3 scripts/validate-tp18.py
actionlint
helm lint devops-tp12/chart -f devops-tp12/values-prod.yaml
helm template tp16 devops-tp12/chart \
  -f devops-tp12/values-prod.yaml > manifests-rendered-prod.yaml
bash scripts/preparar-defectdojo.sh
python3 scripts/inicializar-defectdojo.py
bash scripts/defectdojo-compose.sh ps -a
```

Fase 1: tests de app pasan y existe un tarball con checksum por SHA. Fase 2: cinco herramientas generan seis bundles válidos, con SHA y Run ID coincidentes. Fase 3: base/web/broker saludables, worker responde, API autentica y contexto obtenido del servidor. Fase 4: publicación y deploy sólo si main y todos los controles requeridos aprueban.

La V2 debe indicar parser y archivo exactos: ZAP XML, Threagile risks.json, Semgrep JSON, ocho JSON Trivy y Gitleaks JSON redactado. Un escáner fallido no se sustituye por una lista vacía. No se ocultan errores con || true.

Flujo verificable de importación y reimportación

```
export DEFECTDOJO_URL=http://127.0.0.1:8080
export DEFECTDOJO_TOKEN_FILE="$PWD/.security-private/defectdojo/api-token"
export DEFECTDOJO_PRODUCT_TYPE_ID="$(python3 -c \
 'import json; print(json.load(open(".security-private/defectdojo/context.json"))
["product_type_id"])')"
bash scripts/verificar-defectdojo.sh check
bash scripts/verificar-defectdojo.sh import --bundles RUTA \
  --sha SHA_COMPLETO --run-id RUN_ID
bash scripts/verificar-defectdojo.sh reimport --bundles RUTA \
  --sha SHA_COMPLETO --run-id RUN_ID --output .security-collection/reimport
```

Aceptación: HTTP 201, doce Tests correctos, metadatos consultados y estabilidad de total/IDs/estados al reimportar el mismo conjunto. Una segunda prueba debe variar el Target entre SHAs y registrar cualquier crecimiento real; no debe exigir igualdad ficticia entre conteos del escáner y Dojo.

Conectividad y credenciales en CI

El operador revisa el SHA y el evento del propietario antes de ejecutar el launcher JIT. El job usa environment tp18-defectdojo, Secret DEFECTDOJO_API_KEY y variable del Product Type obtenido de la instancia. PR no confiables quedan excluidos. La aprobación del environment no reemplaza el aislamiento del runner. Cada runner acepta un job y se elimina.

El importador usa always() con éxito explícito del colector y de los tests de reportes. Trivy rojo puede coexistir con importación aprobada. Deploy y publicación siguen requiriendo todos los gates; Dojo nunca revierte un fallo de seguridad.

Persistencia diagnóstico y evidencias

```
bash scripts/respaldar-defectdojo.sh
bash scripts/verificar-defectdojo.sh snapshot --output .security-collection/antes
bash scripts/defectdojo-compose.sh restart postgres valkey uwsgi \
  celeryworker celerybeat nginx
bash scripts/verificar-defectdojo.sh snapshot --output .security-collection/despues
bash scripts/verificar-defectdojo.sh negative --output .security-collection/negativa
git diff --check
```

Diagnosticar en orden: recursos y disco; salud y migraciones; puertos/túnel; token/permisos; contexto; SHA/run/hashes; formato/parser; HTTP; snapshot. Conservar salida no cero ante errores operativos y no imprimir cuerpos sensibles. Para respaldos, proponer ejecución diaria y siete copias; validar restauración primero en una instancia aislada, con las mismas claves y media.

Checklist de entrega: identidad real, versión/contexto, doce Tests, conteos atribuidos a runs, import/reimport, estados antes/después, persistencia, negativos, CI y gates, validaciones, capturas sin secretos y DOCX/PDF revisados. La guía V2 completa queda en docs/TP18-GUIA-V2.md.

5 Interpretación de mensajes reales

Colección válida

```
Collection: 6/6 bundles; missing/invalid=0
```

Lo produjo el cliente al validar los artifacts de la ejecución. Confirma seis bundles con formatos, hashes y procedencia válidos; no significa que no existan vulnerabilidades. Contrasté los manifests y permití continuar a la API.

Importación aceptada

```
OK import-scan: TP18 zap DAST; HTTP 201; Test 1
```

Corresponde a la importación inicial del run 38512049385. El servidor aceptó el XML mediante ZAP Scan y la consulta posterior confirmó el Test y sus 5 alertas. No deduje éxito de una palabra encontrada con grep.

Reimportación estable

```
DefectDojo: 12/12 reports; Tests=12; Findings=469; active=469; duplicates=0
```

El job del run 38664207927 volvió a procesar los doce reportes. Comparé antes/después: 469 → 469, mismos Tests y hash de IDs/estados. El cero de duplicate es una clasificación de Dojo; no elimina la limitación observada entre Targets con distintos SHAs.

Solicitud inválida controlada

```
OK controlled negative test: HTTP 400: POST /api/v2/import-scan/; Tests and Findings unchanged
```

Envié una solicitud inválida intencional. La prueba negativa sólo se considera aprobada si obtiene el 400 esperado y los snapshots siguen iguales. Un error HTTP en una importación normal devuelve salida no cero.

Conectividad fallida

```
ERROR: Network failure: GET /api/v2/products/
```

Apunté temporalmente al puerto loopback cerrado 1. El verificador terminó con exit 1, sin imprimir el token ni asumir éxito. Volví a la URL privada válida y comprobé API/contexto. No fue una caída inventada del servicio real.

Gate Trivy bloqueante

```
Process completed with exit code 1.
```

GitHub registró código 1 en los pasos de Trivy que detectaron HIGH/CRITICAL. Contrasté 61 entradas de la banda: SCA 3, imagen 47 e IaC 11. Conservé el fallo y verifiqué deploy skipped; no correspondía corregir vulnerabilidades para aprobar este TP.

6 Novedades y comportamientos observados

La identidad del hallazgo también depende del contexto

Al comparar importaciones observé que conservar el mismo número de entradas por escáner no garantiza un total estable entre versiones de imagen. Dojo utiliza campos concretos para identificar Findings; el Target con SHA cambió la descripción Trivy y generó 163 registros adicionales. Esta diferencia me llevó a separar idempotencia de reimportación idéntica y deduplicación entre versiones.

Centralizar exige comprobar el ciclo de vida

La primera importación creó doce Tests. La reimportación reutilizó esos IDs y permitió seguir los estados de cada Finding. Comprobar solamente el total habría sido insuficiente: comparé activos, duplicados, aceptados y el hash de IDs/flags. El reinicio conservó la misma información, demostrando persistencia más allá de una pantalla abierta.

Un gate rojo puede acompañar una gestión correcta

Vi que DefectDojo puede importar evidencia real mientras Trivy rechaza la liberación. El colector y el importador pasaron y el deploy permaneció omitido. El resultado global rojo no implica que la centralización haya fallado. También confirmé que el lint histórico TP06 no volvió a bloquear el build de Notes App.

Cero resultados y formatos distintos también son evidencia

Gitleaks produjo una lista vacía válida, aceptada por el parser nativo. No necesité introducir un secreto. En ZAP el XML fue el formato requerido por ZAP Scan, aunque conservara JSON y HTML para revisión. Threagile tuvo un parser propio. Validar JSON sintácticamente no sustituye comprobar el contrato del parser real.

La infraestructura y los permisos forman parte de la prueba

El error Dojo_User mostró una diferencia concreta entre el modelo de usuarios de la aplicación y el User genérico de Django. Ajusté la creación de la cuenta y comprobé su acceso limitado al producto. La conectividad obligó a resolver cómo llega CI a localhost sin abrir HTTP a Internet: un runner efímero revisado fue suficiente para este laboratorio.

También identifiqué que docker compose exec podía consumir la entrada estándar del script SSH. Hice explícito --interactive=false en las operaciones que no reciben datos y repetí las comprobaciones de respaldo y persistencia. La prueba final se basó en sus salidas completas y snapshots, no en la intención de ejecutar los comandos.

7 Conclusión y relación con la teoría

Con TP18 integré la gestión de vulnerabilidades al flujo DevSecOps existente. Las herramientas siguen produciendo evidencia especializada y DefectDojo la reúne bajo un producto, un Engagement y Tests trazables. La centralización aporta un lugar persistente para comparar estados y planificar seguimiento; no convierte automáticamente una alerta en riesgo confirmado ni aplica una remediación.

Relación con los controles de seguridad

Shift Left aparece en el hook Gitleaks y los controles tempranos de código e infraestructura. Semgrep aporta SAST; ZAP observa respuestas del backend en ejecución como DAST. Trivy cubre SCA, imagen e IaC, y conserva su gate HIGH/CRITICAL. Gitleaks recorre el historial Git para Secret Scanning. Threagile agrega Threat Modeling y riesgos de diseño que no dependen de ejecutar la aplicación.

Estos controles cubren superficies diferentes. Mantener cinco herramientas evita depender de una sola vista del sistema. El build único y los hashes vinculan auditorías a un artefacto concreto; el SHA, Run ID y los comprobantes API extienden esa trazabilidad hasta los Findings centralizados.

Riesgo automatización y responsabilidades

La severidad técnica permite ordenar trabajo, pero debe combinarse con exposición, impacto de negocio y revisión humana. Los tres riesgos aceptados de Threagile reflejan el modelo heredado, no una aceptación nueva que otorgué para aprobar CI. No marqué como verificados hallazgos sin análisis manual.

Automaticé la importación, la reimportación y la comparación de estados con errores explícitos. La defensa en profundidad incluye la restricción de red, la cuenta limitada al producto, el environment protegido, la validación de procedencia y la separación entre el importador y los gates de release. La aprobación del importador nunca autoriza por sí sola la publicación.

Infraestructura debe mantener capacidad, backups, claves, conectividad y actualizaciones de la instancia. Seguridad y desarrollo deben revisar findings, priorizar riesgos y documentar decisiones. El operador del laboratorio debe revisar y lanzar cada runner JIT; esta asistencia deliberada es una limitación operativa del modelo elegido.

Límites y mejora continua

Comprobé persistencia y estructura de backup, pero falta ensayar una restauración completa aislada y automatizar su política. No implementé alta disponibilidad ni TLS directo; el acceso HTTP permanece en loopback a través de SSH. ZAP no cubre frontend, Ingress/TLS ni login; Semgrep conserva una limitación PartialParsing en Bash.

Como pasos futuros propongo evaluar deduplicación por identidad estable sin perder procedencia, verificar restauración y ampliar cobertura DAST. Si apareciera un secreto real, primero correspondería revocar o rotar la credencial y después evaluar el saneamiento del historial de forma coordinada. No ejecuté reescrituras ni remediaciones en este trabajo.

Demostración breve para la defensa en clase

1. Abrir la terminal en ~/tp18 y mostrar whoami, hostname, git remote -v y el commit técnico 84088cb. Identificar la rama y el run que respaldan el informe.

2. Ejecutar bash scripts/defectdojo-compose.sh ps -a. Explicar initializer Exited 0, los seis servicios activos y las tres persistencias. Acceder a la UI mediante el túnel SSH privado.

3. Abrir Notes App y el Engagement CI/CD Automated Security Audits. Mostrar los doce Tests, las cinco herramientas y un Finding. Explicar por qué Gitleaks tiene cero y por qué los 469 registros no son 469 vulnerabilidades únicas.

4. Mostrar los comprobantes de importación y reimportación del run 38664207927. Comparar 469 → 469, mismos IDs/estados, y la persistencia después del reinicio. Si se repite la prueba, usar los mismos bundles, SHA y Run ID válidos.

5. Abrir GitHub Actions: colector e importador aprobados, Trivy rojo y deploy skipped. Relacionar ese resultado con Andon Cord y las cuatro fases. No lanzar otro run sin revisar y habilitar su runner JIT.

Archivos que permiten reproducir y revisar

README.md y docs/TP18-PREPARACION.md describen el estado; docs/TP18-DEFECTDOJO.md detalla instalación, acceso, permisos, API y respaldo; docs/TP18-GUIA-V2.md contiene el procedimiento propuesto. infra/defectdojo/ fija el stack; scripts/defectdojo_api.py y verificar-defectdojo.sh implementan las operaciones. entrega/capturas conserva los originales y entrega/evidencias registra resultados sanitizados.

Referencias utilizadas

Guía de la cátedra: TP18 DefectDojo Vulnerability Management, documento original entregado para el trabajo. Consulté también las siguientes fuentes oficiales para la versión instalada y el modelo de ejecución. Fecha de consulta: 14 de octubre de 2026.

DefectDojo CE 3.4.101 y release oficial

Compose oficial versionado

Instalación CE

API v2 serializers de 3.4.101

Parser Trivy nativo de 3.4.101

GitHub seguridad de Actions y runners

GitHub runners propios